

Chapter 1

Vectors

In the previous chapter we used a loop to compute the elements (scalars) of a sequence, but we were only able to store the last element. In this chapter, we'll use a vector to store all the elements in an array. We'll also learn how to select elements from a vector, and how to perform vector arithmetic and common vector operations like *reduce* and *apply*.

1.1 Creating Vectors

A *vector* in MATLAB is a sequence of numbers, stored as an array. There are several ways to create vectors; one of the most common is to put a sequence of numbers in square brackets:

```
>> [1 2 3]
ans = 1      2      3
```

The `size` function tells you the number of elements in each dimension.

```
>> size(xx)
ans =
     1     3
```

This indicates that `xx` is a 1-by-3 array. We will learn later that this is a *row* vector.

Another way to create a vector is the colon operator, which we have already used to create a range of values in a `for` loop.

```
>> 1:3
ans = 1      2      3
```

In general, anything you can do with a number, you can also do with a vector. For example, you can assign a vector value to a variable:

```
>> xx = [1 2 3]
X = 1      2      3
```

Note that variables that contain vectors are often double letters (**xx**) or capital letters (**X**). These are just conventions; MATLAB doesn't require it, but it's a useful way to remember which variables are vectors.

Here are the syntax for some typical ways to create a vector:

```
% Some ways to initialize vectors
% As an list with step size = 1
1:5
ans =
     1     2     3     4     5

% As a list with step size ~= 1
1:3:10
ans =
     1     4     7    10

% As an N-by-1 column of zeros
zeros(5,1)
ans =
     0
     0
     0
     0
     0

% As a 1-by-N row of ones
ones(1,5)
ans =
     1     1     1     1     1

% A vector of psuedorandom numbers
rand(1,5)
ans =
    0.1576    0.9706    0.9572    0.4854    0.8003

% Manually, as a row vector
[5 3.1 9.8 -2]
ans =
    5.0000    3.1000    9.8000   -2.0000

% Manually as a column vector
[5; 3.1; 9.8; -2;]
ans =
    5.0000
    3.1000
```

```

    9.8000
   -2.0000

% As a column using transpose
[5 3.1 9.8 -2]'
ans =
    5.0000
    3.1000
    9.8000
   -2.0000

% Linear spacing from A to B with N elements
linspace(0, 1, 5)
ans =
    0    0.2500    0.5000    0.7500    1.0000

% Logrithmic spacing from A to B with N elements
logspace(0, 1, 5)
ans =
    1.0000    1.7783    3.1623    5.6234   10.0000

```

1.2 Vector Arithmetic

As with numbers, you can do arithmetic with vectors. However, operations with vectors can be done in more than one way, so to avoid logical errors, we need to be more intentional with the instructions we provide. In general, MATLAB executes both *array operations* and *matrix operations*. Matrix operations follow the rules of linear algebra, while array operations execute element by element. The syntax for each can be closely related.

If you add a scalar to a vector, MATLAB does the array operation increments each element of the vector:

```

>> Y = X + 5
Y = 6    7    8

```

The result is a new vector; the original value of **X** has not changed.

If you add two vectors, MATLAB adds the corresponding elements of each vector and creates a new vector that contains the sums:

```

>> Z = X + Y
Z = 7    9   11

```

But adding vectors only works if the operands are the same size. Otherwise, you get an error:

```

>> W = [1 2]

```

```
W = 1      2
```

```
>> X+W
```

```
Arrays have incompatible sizes for this operation.
```

```
Related documentation
```

The `size()` function can be helpful in debugging runtime errors of vector/matrix sizing, e.g.,

```
>> size(X)
```

```
ans =
```

```
    1    3
```

```
>> size(W)
```

```
ans =
```

```
    1    2
```

might help us investigate why we received the ‘Matrix dimensions must agree’ error.

If you divide two vectors, you might be surprised by the result:

```
>> X / Y
```

```
ans = 0.2953
```

MATLAB is performing a matrix operation called right-matrix division, which is not what we expected. (See the `mrdivide()` function documentation for more details.) If, instead, we wanted MATLAB to do the array operation of divide the elements of `X` by the elements of `Y`, you have to use `./`, which is element-wise division:

```
>> X ./ Y
```

```
ans = 0.1667    0.2857    0.3750
```

Multiplication has the same problem. If you use `*`, MATLAB does matrix multiplication following the rules of linear algebra. With these two vectors, matrix multiplication is not defined, so you get an error:

```
>> X * Y
```

```
Error using *
```

```
Incorrect dimensions for matrix multiplication.
```

```
Check that the number of columns in the first matrix  
matches the number of rows in the second matrix.
```

```
To perform element-wise multiplication, use '.*'.
```

In this case, the error message is pretty helpful. As it suggests, you can use `.*` to perform element-wise multiplication:

```
>> X .* Y
```

```
ans = 6    14    24
```

As an exercise, see what happens if you use the exponentiation operator (`^`) with a vector.

1.3 Indexing and Slicing

In MATLAB, array *indexing* refers to accessing specific elements of an array using their position, while *slicing* allows selecting multiple elements, such as a range of rows, columns, or both. MATLAB uses 1-based indexing, meaning the first element is at position 1. To access an element, you specify its index in parentheses. For example, for vector `Y`:

```
>> Y = [6 7 8 9]
Y = 6     7     8     9

>> Y(1)
ans = 6

>> Y(4)
ans = 9
```

This means that the first element of `Y` is 6 and the fourth element is 9. The number in parentheses is called the *index* because it indicates which element of the vector you want.

The index can be a variable name or a mathematical expression:

```
>> i = 1;
>> Y(i)
ans = 6
>> Y(i+1)
ans = 7
```

We can use a loop to display the elements of `Y`:

```
for i=1:4
    Y(i)
end
```

Each time through the loop we use a different value of `i` as an index into `Y`.

In the previous example we had to know the number of elements in `Y`. We can make it more general by using the `length` function, which returns the number of elements in a vector:

```
for i=1:length(Y)
    Y(i)
end
```

This version works for a vector of any length.

When slicing a vector, you can extract a subset of the array using colon notation `:` to indicate ranges.

```
>> B = [1, 2, 3, 4, 5]
B =
     1     2     3     4     5
>> C = B(2:4)
```

```
C =
     2     3     4
```

You can also slice a vector to extract the elements described by another vector:

```
>> D = B([2 5])
D =
     2     5
```

1.4 Indexing Errors

An index can be any kind of expression, but the value of the expression has to be a positive integer, and it has to be less than or equal to the length of the vector. If it's zero or negative, you'll get an error:

```
>> Y(0)
Array indices must be positive integers or logical values.
```

If it's not an integer, you get an error:

```
>> Y(1.5)
Array indices must be positive integers or logical values.
```

If the index is too big, you also get an error:

```
>> Y(5)
Index exceeds the number of array elements (4).
```

The `end` keyword can help with indexing and slicing. It refers to the maximum index value, so that

```
>> Y(end)
ans =
     8
```

will index the last element in the vector no matter how many elements are included. This is particularly helpful when slicing the last N elements of a vector:

```
>> G = 1:10
G =
     1     2     3     4     5     6     7     8     9    10
>> N = 5
N =
     5
>> H = G(end-N:end)
H =
     5     6     7     8     9    10
```

1.5 Vectors and Sequences

Vectors and sequences go together nicely. For example, another way to evaluate the Fibonacci sequence from Chapter 2 is to store successive values in a vector. Remember that the definition of the Fibonacci sequence is $F_1 = 1$, $F_2 = 1$, and $F_i = F_{i-1} + F_{i-2}$ for $i > 2$.

Listing 1.1 shows how we can compute the elements of this sequence and store them in a vector, using a capital letter for the vector **F** and lower-case letters for the integers **i** and **n**.

Listing 1.1: Calculating the Fibonacci sequence using a vector

```
F(1) = 1
F(2) = 1
for i=3:n
    F(i) = F(i-1) + F(i-2)
end
```

If you had any trouble with `,`, you have to appreciate the simplicity of this script. The MATLAB syntax is similar to the math notation, which makes it easier to check for correctness.

If you only want the n th Fibonacci number, storing the whole sequence wastes some space. But if wasting space makes your code easier to write and debug, that's probably okay.

1.6 Plotting Vectors

If you call `plot` with a vector as an argument, MATLAB plots the indices on the x-axis and the elements on the y-axis. To plot the Fibonacci numbers we computed in Listing 1.1, we'd use

```
plot(F)
xlabel('Index [n/a]')
ylabel('Fibonacci number [n/a]')
```

Figure 1.1 shows the result.

This way of looking at a vector is often useful for debugging, especially if it is big enough that displaying the elements on the screen is unwieldy.

1.7 Common Vector Operations

We've covered some basic features of vectors. Let's now look at some common patterns we use to work with data stored in vectors.

1.7.1 Reduce

We frequently use loops to run through the elements of a vector and add them up, multiply them together, compute the sum of their squares, and so on. This kind of operation is called *reduce*, because it reduces a vector with multiple elements down to a single number.

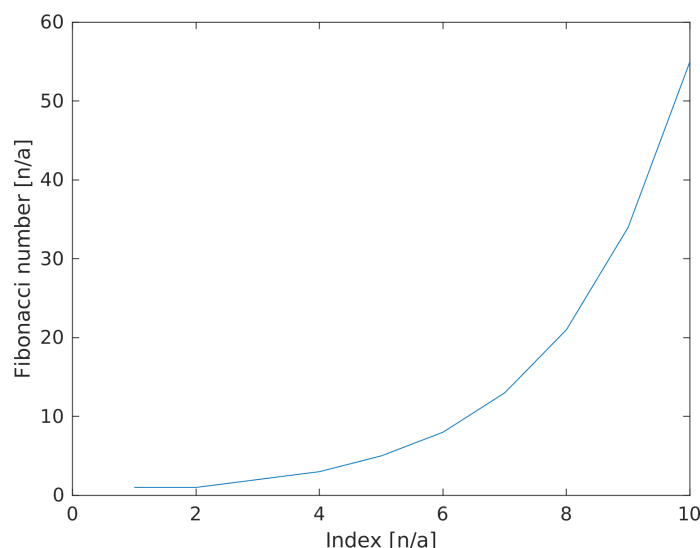


Figure 1.1: The first 10 elements of the Fibonacci sequence

For example, the loop in Listing 1.2 adds up the elements of a vector named **X** (which we assume has been defined).

Listing 1.2: Reducing a vector to a single scalar value (the sum)

```
total = 0
for i=1:length(X)
    total = total + X(i)
end
ans = total
```

The use of **total** as an accumulator is similar to what we saw in Chapter 3. Again, we use the **length** function to find the upper bound of the range, so this loop will work regardless of the length of **X**. Each time through the loop, we add in the *i*th element of **X**, so at the end of the loop **total** contains the sum of the elements.

MATLAB provides functions that perform some reduce operations. For example, the **sum** function computes the sum of the elements in a vector, and **prod** computes the product.

1.7.2 Apply

Another common use of a loop is to run through the elements of a vector, perform some operation on the elements, and create a new vector with the results. This operation is called *apply*, because you apply the operation to each element in the vector.

For example, the loop in Listing 1.3 creates a vector **Y** that contains the squares of the elements of **X** (assuming, again, that **X** is already defined).

Listing 1.3: Making a new vector **Y** by squaring the elements in **X**

```
for i=1:length(X)
```

```
Y(i) = X(i)^2  
end
```

Many apply operations can be done with element-wise operators. The following statement is more concise than the loop in Listing 1.3.

```
Y = X .^ 2
```

It also runs faster!

1.8 Chapter Review

In this chapter, we used a vector to store the elements of a sequence. We learned how to select elements from a vector and perform vector arithmetic. We performed reduce and apply operations using `for` loops, MATLAB functions, and element-wise operations.

Here are some terms from this chapter you might want to remember.

A *vector* is a sequence of values, which is a kind of *matrix*, also called an *array* in some MATLAB documentation.

An *index* is an integer value used to indicate one of the elements in a vector or matrix (also called a *subscript* in some MATLAB documentation).

An operation is *element-wise* if it acts on the individual elements of a vector or matrix (unlike some linear algebra operations).

You can *apply* an operation to all elements of a vector, and you can *reduce* a vector to a single value, for example by computing the sum of the elements.

In the next chapter, we'll meet the most important idea in computer programming: functions!

1.9 Exercises

Before you go on, you might want to work on the following exercises.

Exercise 1.1. (From 18.S997 OCW.)

Practice creating vectors in various ways.

- Try out sequences with step-size other than 1, e.g. `[4:0.1:5]`, `[5:-2:-5]`.
- Create a list of the whole numbers between 10 and 20 (inclusive), find their sum.
- Create the vector of the previous question in decreasing order.
- Find the sum of the odd numbers between 100 and 200.

Exercise 1.2. Write a loop that computes the first n elements of the geometric sequence $A_{i+1} = A_i/2$ with $A_1 = 1$. Notice that math notation puts A_{i+1} on the left side of the

equality. When you translate to MATLAB, you might want to rewrite it with A_i on the left side.

Exercise 1.3. Write an expression that computes the square root of the sum of the squares of the elements of a vector, without using a loop.

Exercise 1.4. The ratio of consecutive Fibonacci numbers, F_{n+1}/F_n , converges to a constant value as n increases. Write a script that computes a vector with the first n elements of a Fibonacci sequence (assuming that the variable `n` is defined) and then computes a new vector that contains the ratios of consecutive Fibonacci numbers. Plot this vector to see if it seems to converge. What value does it converge on?

Exercise 1.5. The following set of equations is based on a famous example of a chaotic system, the Lorenz attractor (see <https://greenteapress.com/matlab/lorenz>):

$$\begin{aligned}x_{i+1} &= x_i + \sigma(y_i - x_i) dt \\y_{i+1} &= y_i + [x_i(r - z_i) - y_i] dt \\z_{i+1} &= z_i + (x_i y_i - b z_i) dt\end{aligned}$$

1. Write a script that computes the first 10 elements of the sequences X , Y , and Z and stores them in vectors named `X`, `Y`, and `Z`.

Use the initial values $X_1 = 1$, $Y_1 = 2$, and $Z_1 = 3$ with the values $\sigma = 10$, $b = 8/3$, $r = 28$, and $dt = 0.01$.

2. Read the documentation for `plot3` and `comet3`, and plot the results in three dimensions.
3. Once the code is working, use semicolons to suppress the output and then run the program with sequence lengths of 100, 1,000, and 10,000.
4. Run the program again with different starting conditions. What effect does it have on the result?
5. Run the program with different values for σ , b , and r , and see if you can get a sense of how each variable affects the system.

Exercise 1.6. The logistic map (see <https://greenteapress.com/matlab/logistic>) is described by the following equation:

$$X_{i+1} = rX_i(1 - X_i)$$

where X_i is a number between 0 and 1, and r is a positive number.

1. Write a script named `logmap.m` that computes the first 50 elements of X with `r` = 3.9 and `X1` = 0.5, where `r` is the parameter of the logistic map and `X1` is the initial value.
2. Plot the results for a range of values of r from 2.4 to 4.0. How does the behavior of the system change as you vary r ?

Exercise 1.7. For this exercise write a live script named *firstorder.mlx* to examine the connection between a geometric series (progression) and the time response of a first-order system.

A first-order, linear ordinary differential equation (ODE) is a common mathematical model useful to predict the behavior of many engineering systems. A typical form for a system undergoing decay is

$$\frac{dy}{dt} = -k y(t)$$

where $y(t)$ is the quantity that decays over time, k is the constant decay rate and t is time. The solution to this ODE is

$$y(t) = y_0 e^{-kt}$$

where y_0 is the initial condition.

Computers, and computer programs, don't have a way to handle such this continuous time function directly. One thing we can do is to sample the function at discrete points and plot the results.

- Write text block and code block to illustrate a specific example of the time response, $y(t)$, where $y_0 = 20$ units and $k = 2\text{ s}^{-1}$. Use the `linspace` command to create a time vector of length 100, starting at 0 and ending at 4. Plot the first-order decay as $y(t)$ vs time.

In computational solutions, we often approximate continuous processes using discrete time steps because computers operate with finite, step-by-step calculations. They cannot directly handle continuous, infinitely varying data. We can approximate the above ODE with the following *difference equation*:

$$y[n] = y[n-1](1 - k\Delta t)$$

where $y[n]$ is the value at the n -th time step, $y[n-1]$ is the value at the previous time step and Δt is the time step between consecutive points in the discrete-time system. This is the interval over which we are updating the system's state from one step to the next.

- Add text and code plots to implement the recurrent, discrete time approximation. Start with a small time step, say $\Delta t = 0.04\text{ s}$. Plot the discrete time approximation on the same axes you used for the sampled analytical solution and use a discrete marker to show each element in the solution.
- Experiment with increasing the time step to see how it affects the approximation. Add a text block to the live script to describe the effect of the time step on the accuracy of the discrete time approximation.